```
synchronisedInterval = (fn, t, maxDrift) ->
  drift = null
  previous = null
  compensate = 5
  reset = (hard = false) ->
    drift = 0 if hard
    previous = Date.now()
    if drift > maxDrift
      setTimeout runner, (t - compensate)
    else
      setTimeout runner, t
  runner = ->
    current = Date.now()
  drift += current - previous - t
  previous = current
  fn()
  reset()
reset true
```

The compensation is achieved by resetting the timeout, accounting for the difference between the expected previous timeout and the actual timeout.

If you have 100 timeouts running and you're trying to do drift compensation for all of them, then you might notice the browser performance starting to degrade. One solution in cases where you need to synchronize many events is to have a single *world* interval that synchronizes all the other events in the program to it.

Understanding different techniques for drawing, animation, and timing is essential to writing effective browser applications. Although programming for browser-based user interfaces is a topic too big to cover fully here, you have learned the core concepts you need to start building interfaces in CoffeeScript today.

11.7. Summary

In this chapter you learned about writing CoffeeScript programs in a web browser and the unique challenges that browser incompatibilities, animations, and timing present to you as a developer.

In the next chapter you'll see how to put together the various server and client components of your CoffeeScript application in a reliable and repeatable way with modules and builds.